UADE

UADE

Programación III – Diseño y Análisis de Algoritmos

Temas a desarrollar

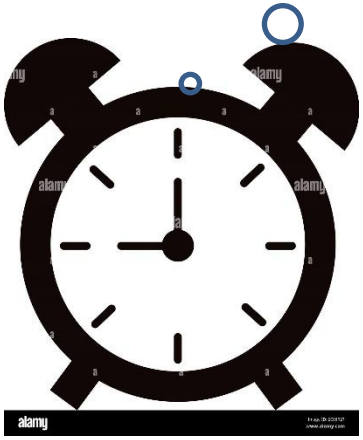
1. Repaso clase anterior
2. Resolución de Ejercicios Greedy
3. Dijkstra
4. Árboles de Descubrimiento

Próxima Clase Remota

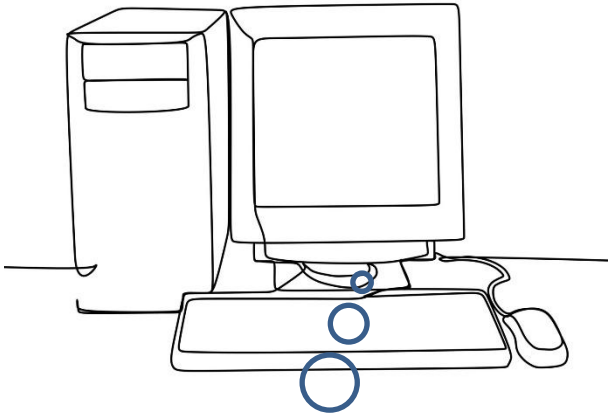
UADE

Próxima Clase Remota

Sábado 05/9 a las 9hrs









La clase es remota por Teams, a conectarse!

UADE

Repaso clase Anterior

UADE

Síntesis Código Huffman

- Los códigos de Huffman se usan para comprimir información eficientemente, logrando una reducción del 20% al 90% (dependiendo de la información original).

- La información es representada como una secuencia de caracteres, donde cada carácter tiene una frecuencia conocida. Suponemos que cada carácter es representado en binario.

- El objetivo del problema es obtener la codificación de menor tamaño.

- Para llevar a cabo la estrategia de solución por *Greedy* se construye un árbol binario comenzando por las hojas, de los símbolos a codificar. Luego se va construyendo de abajo hacia arriba. Para cada símbolo, la ruta desde la raíz proporciona la codificación.

Síntesis Código de Huffman

Codificación variable de un texto

- **Conjunto de candidatos**: los símbolos a utilizar para la codificación de la palabra.
- **La función selección**: se comenzarán a trabajar con los símbolos con menor frecuencia.
- **La función factibilidad**: en este caso particular no habría una limitante a validar.
- **La función solución**: haber construido la codificación (árbol de Huffman).
- **La función objetivo**: minimizar el tamaño de la codificación.

La complejidad temporal de la estrategia desarrollada es de $\Theta(n \log n)$, siendo n la cantidad de símbolos.

Revisión Ejercicios Prácticos

UADE

Ejercicios a Resolver

a) Estrategia de la solución aplicando Greedy e indicando cada uno de los elementos que se requieren para la resolución de esa técnica.
b) Pseudocódigo del Algoritmo de la resolución del problema, el cual debe seguir la estrategia definida en el punto a)
c) Cálculo de la Complejidad Temporal, justificando la misma en forma detallada.

3) **Mezclado de Cintas:** Dado un conjunto de **n** cintas con n_i registros ordenados en cada una de ellas. se pretende mezclarlas de a pares hasta lograr una única cinta ordenada. La secuencia en la que se realiza la mezcla determinará la eficiencia del proceso. Diseñar un algoritmo que busque la solución óptima minimizando el número de movimientos.

Por ejemplo:
Se tienen 3 cintas: A con 30 registros, B con 20 y C con 10, si analizamos dos posibles opciones de mezcla obtenemos:

A. Mezclamos A con B (50 movimientos) y el resultado con C (60 movimientos), con lo que realizamos en total 110 movimientos.

B. Mezclamos C con B (30 Movimientos) y el resultado con A (60). Total = 90 movimientos

Se puede observar que en la opción B. se tienen menos movimientos.

Ejercicios a Resolver

a) Estrategia de la solución aplicando Greedy e indicando cada uno de los elementos que se requieren para la resolución de esa técnica.
b) Pseudocódigo del Algoritmo de la resolución del problema, el cual debe seguir la estrategia definida en el punto a)
c) Cálculo de la Complejidad Temporal, justificando la misma en forma detallada.

4) **Problema del Mecánico:** Un mecánico necesita llevar a cabo **n** reparaciones urgentes conociendo el tiempo que le va a llevar cada una de ellas, es decir, en la tarea i-ésima tardará t_i minutos. Debido a que el pago que recibe el mecánico depende del nivel de satisfacción del cliente, necesita decidir el orden en el que atenderá cada reparación para minimizar el tiempo medio de espera de los clientes. Es decir, si llamamos E_i a lo que espera el cliente i-ésimo hasta ver finalizada su reparación, se necesita minimizar la expresión:

$$T(n) = \sum_{i=1}^{n} E_i$$

Dijsktra

Grafos

Dirigidos: aquellos grafos en donde las aristas tienen un sentido, es decir, no es lo mismo la conexión entre el nodo A y el nodo B, que la conexión entre el nodo B y el nodo A



No dirigidos: aquellos grafos en donde las aristas no tienen, es decir, es lo mismo la conexión entre el nodo A y el nodo B, que la conexión entre el nodo B y el nodo A



Greedy Grafos - Dijsktra

Problema de los caminos mínimos:

> *Dado un grafo y un nodo particular del mismo, buscar el costo del camino mas corto entre ese nodo y todos los demás, siempre que exista un camino.*

El resultado es un grafo con los mismos nodos que el original pero solo hay aristas entre el nodo buscado y todos los demás y el peso de la arista es el costo del camino mínimo.

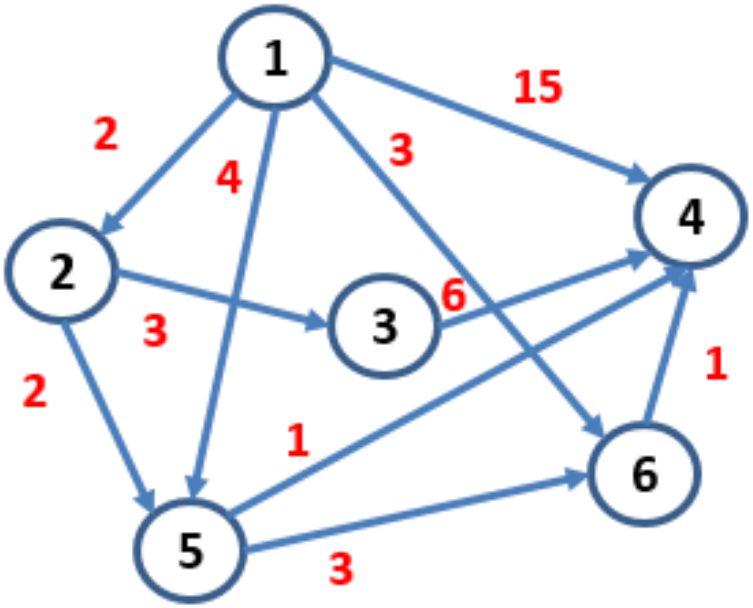
Para poder ejecutar el algoritmo es necesario contar con un grafo dirigido, con todas sus aristas con peso positivo.

Greedy Grafos – Dijsktra - Estrategia

- ***Candidatos***: El conjunto de candidatos es el conjunto de vértices/nodos del grafo, sacando el nodo de origen.
- ***La función selección:*** En cada iteración, el método de selección será el encargado de elegir de todos los nodos pendientes de visitar, aquel que tenga el camino de menor costo desde el origen y lo marca como visitado, además de confirmar como definitivo el costo del camino mínimo desde el origen hasta dicho nodo.
- ***La función factibilidad:*** Calcular que si existe un camino directo el costo sea mayor a pasar por el nodo seleccionado, en caso de que exista camino pasando por ese nodo.
- ***La función solución****: Haber trabajado con todos los nodos candidatos*
- ***La función objetivo:*** Grafo con el valor de todos los caminos mínimos desde el nodo origen dado.

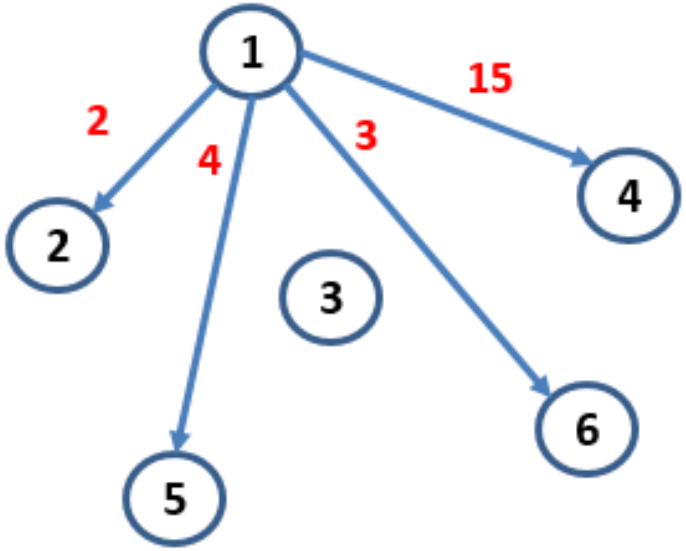
Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Calcular el camino mínimo desde el vértice “1” al resto

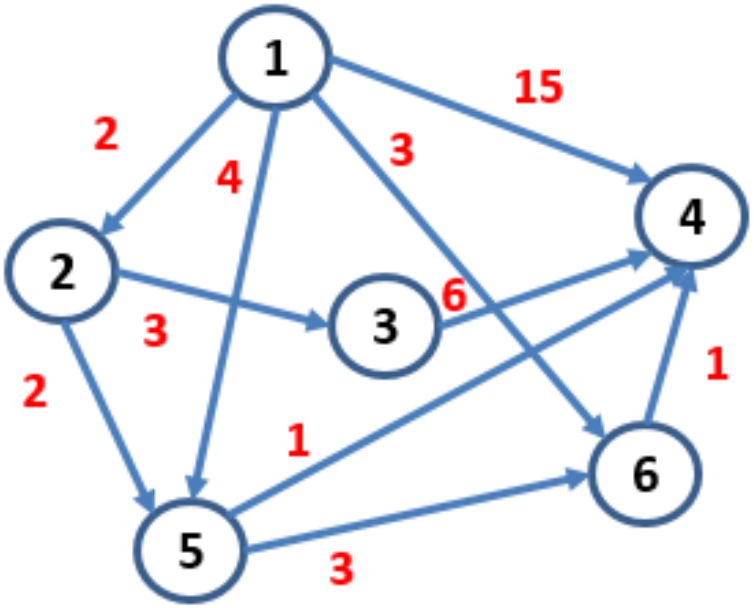
Grafo Resultado



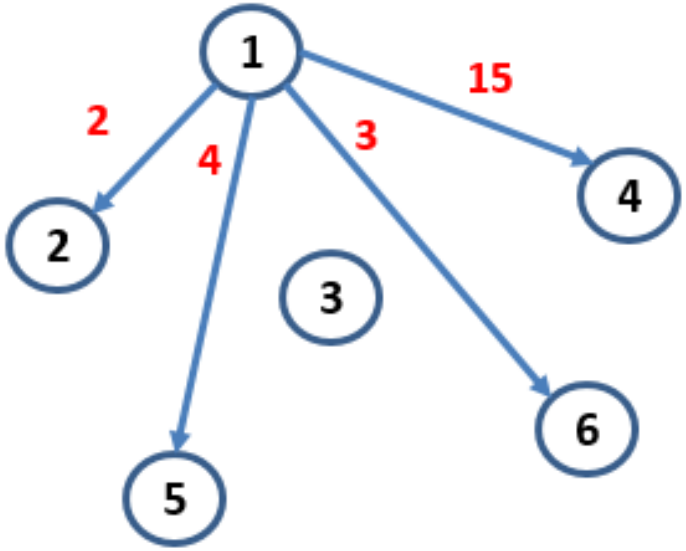
UADE

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado



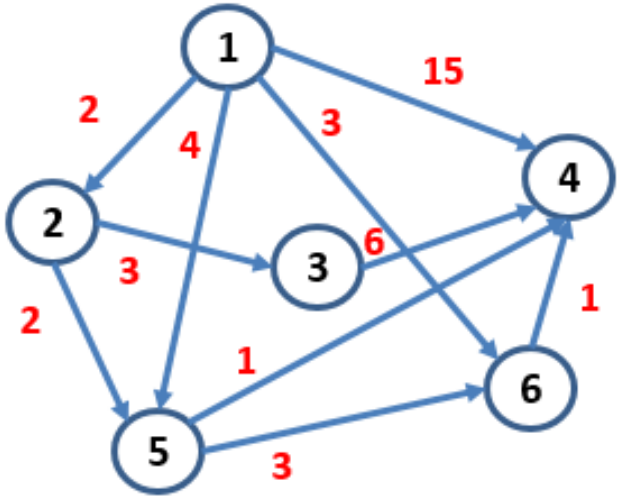
Cantidatos desde "1" = {2, 3, 6, 5, 4}

Se selecciona el más cercano que es el nodo "2" por tener el peso menor "2"

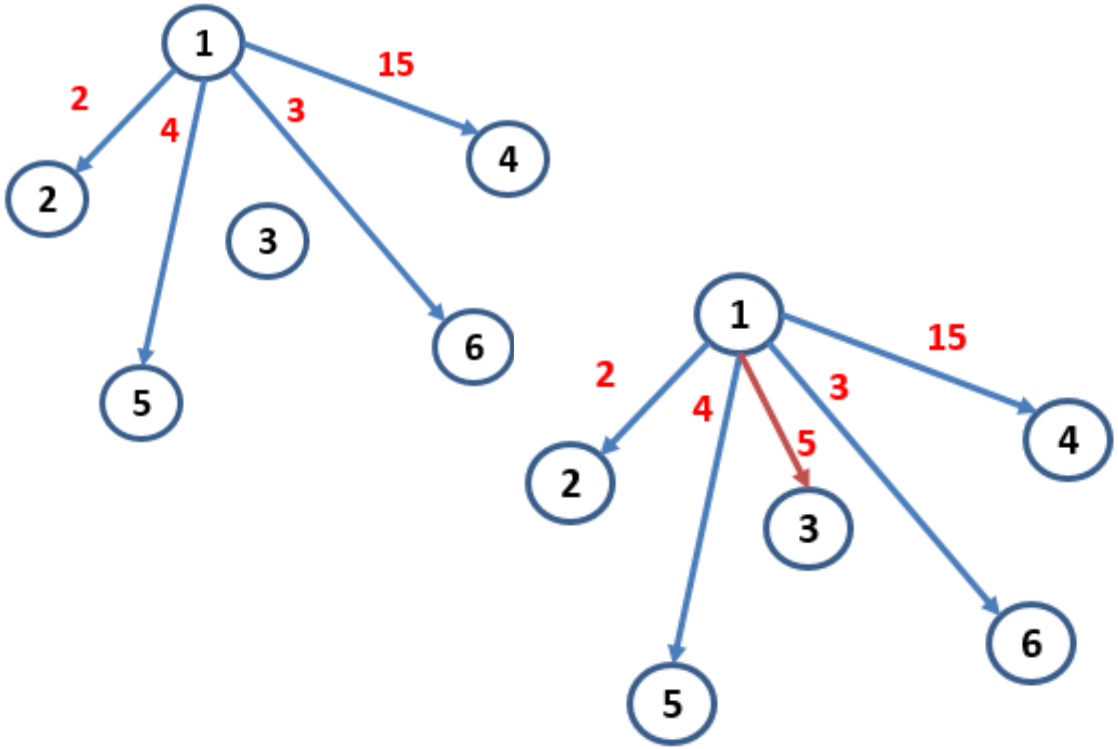
Recalculamos los caminos desde "1" al resto de los nodos pasando por "2" los caminos:

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado



Camino desde "1" a "5" directo es 4, pasando por "2" el costo es "4" también por lo tanto no mejora

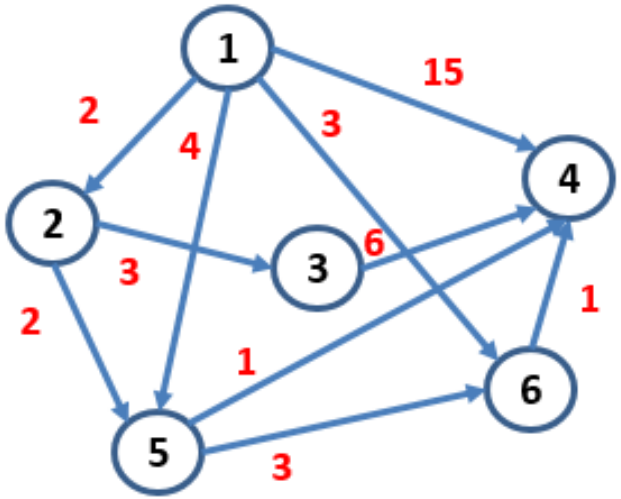
Camino desde "1" a "3" directo no existe, pasando por "2" el costo es "5" el cual es mejor a no existir

Camino desde "1" a "6" directo es 3, pasando por "2" no existe, queda el que está

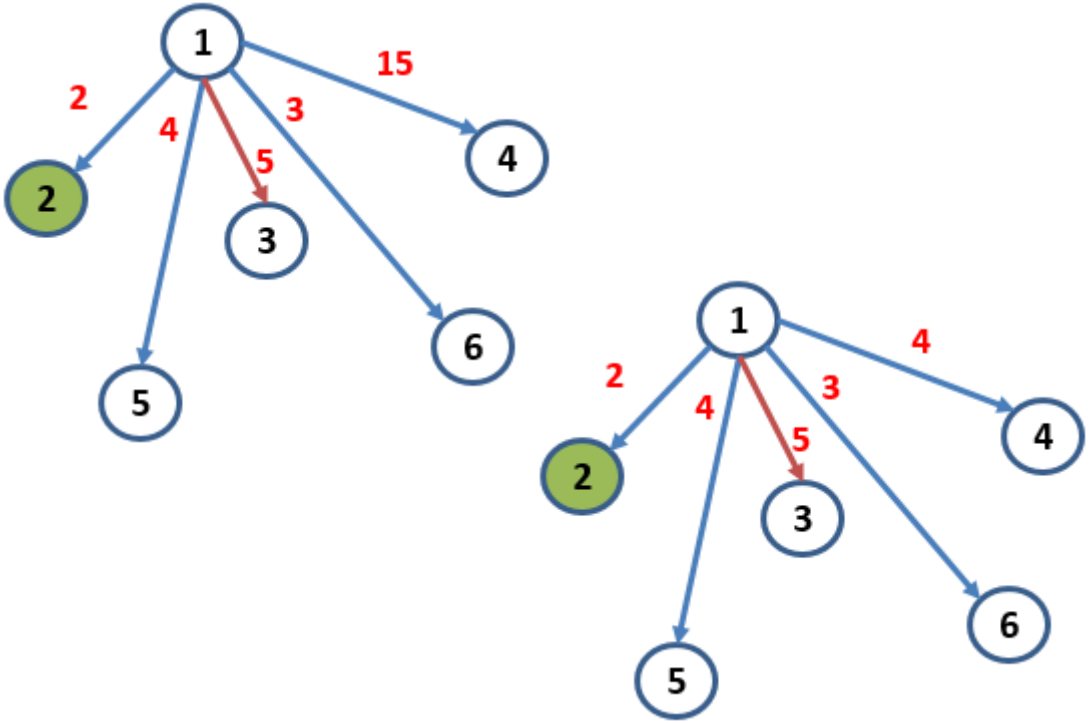
Camino desde "1" a "4" directo es 15, pasando por "2" no existe, queda el que está

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado



Se selecciona el más cercano que es el nodo "6" por tener el peso menor "3" entre los que quedan

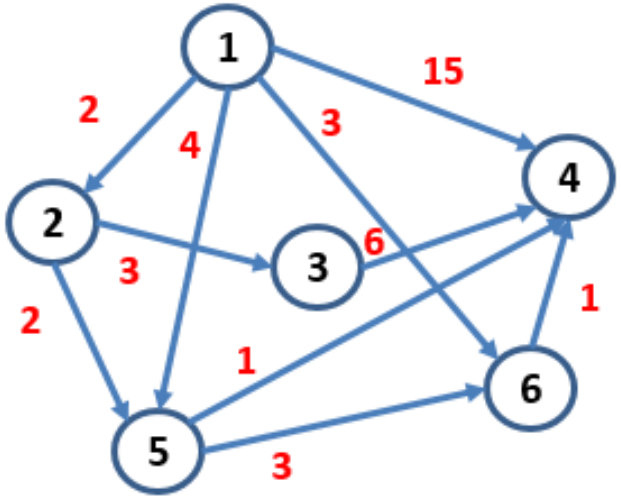
Camino desde "1" a "5" directo es 4, pasando por "6" no existe, queda el que está

Camino desde "1" a "3" directo es 5, pasando por "6" no existe, queda el que está

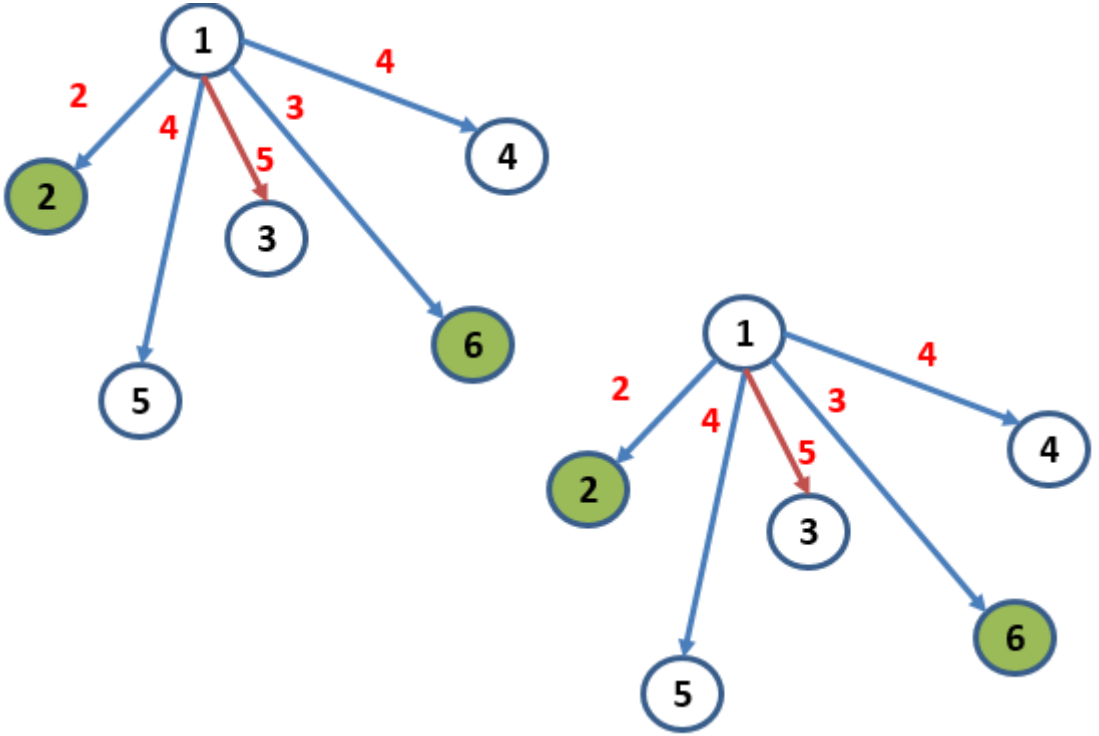
Camino desde "1" a "4" directo es 15, pasando por "6" el costo es "4", por lo cual mejora y se actualiza

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado



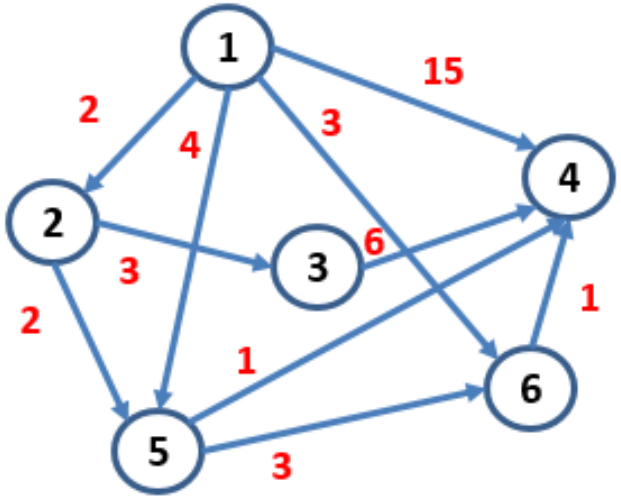
Se selecciona el más cercano que es el nodo "5" ó "4" por tener el peso menor "4", es indistinto cual, vamos a seleccionar 5.

Camino desde "1" a "3" directo es 5, pasando por "5" no existe, queda el que está

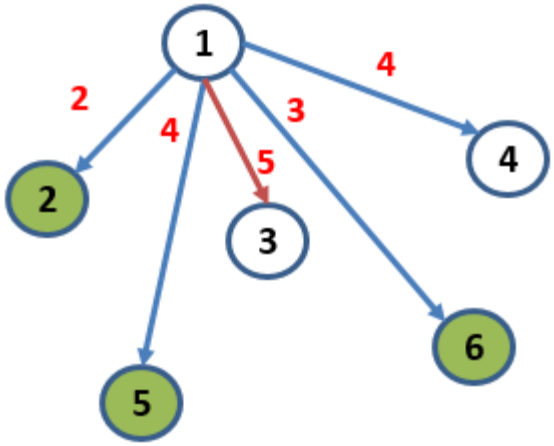
Camino desde "1" a "4" directo es 4, pasando por "5" el costo es "5", por lo cual no mejora, se deja el que esta

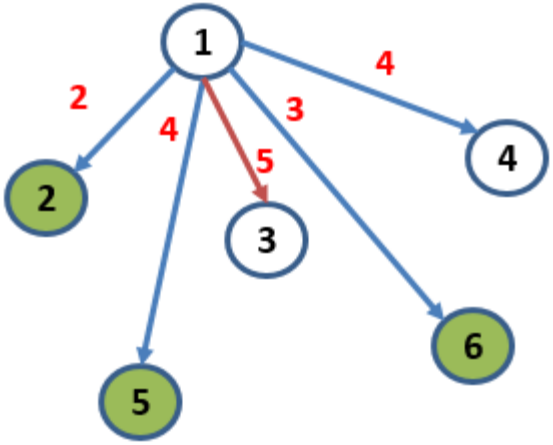
Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado





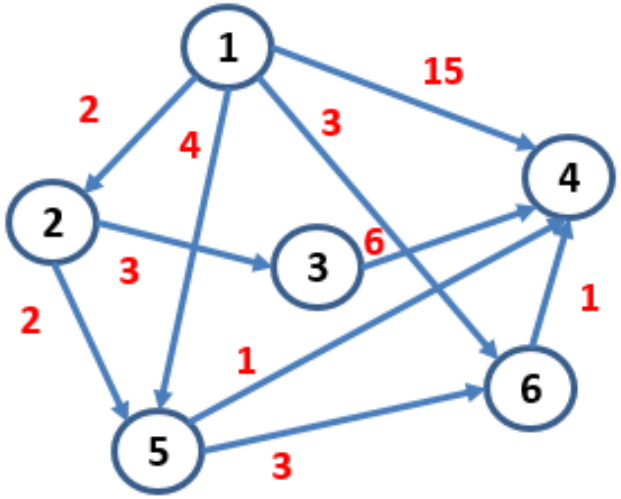
Se selecciona el más cercano que es el nodo "4" por tener el peso menor "4"

Camino desde "1" a "3" directo es 5, pasando por "4" no existe, queda el que está

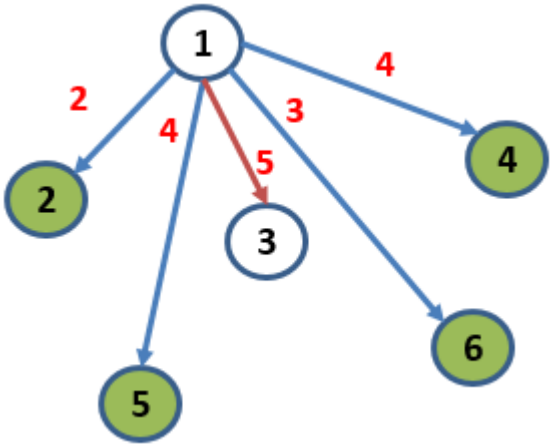
UADE

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado



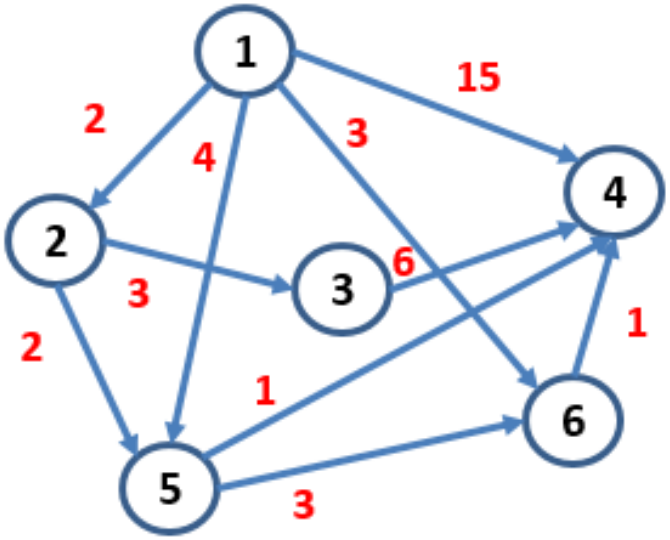
Se selecciona el más cercano que es el nodo "3" por tener el peso menor "5"

No quedan nodos por los cuales pasar, por lo tanto, se obtuvo el resultado final

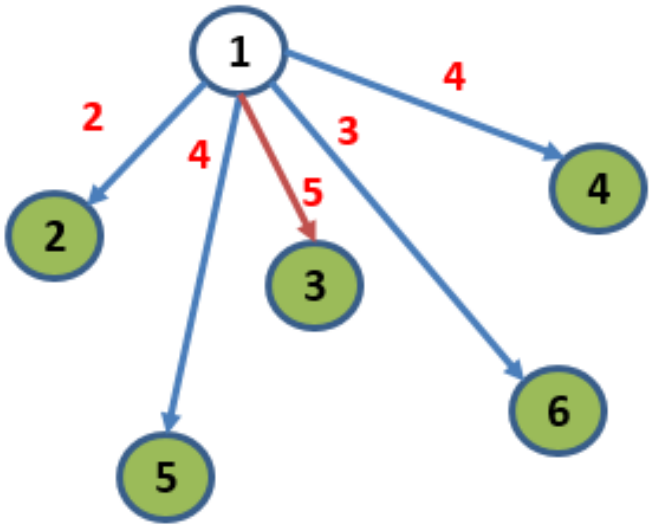
UADE

Greedy Grafos – Dijsktra - Ejemplo

Grafo Original



Grafo Resultado FINAL



UADE

Greedy Grafos – Dijsktra - Algoritmo

ALGORITMO DIJKSTRA

Entrada: G: Grafo<entero>, v: entero
Salida: A Grafo<entero>

```
//Paso 1 //Conjunto de Vértices ya visitados
```
Conjunto<entero> $Visitados \leftarrow \{v\}$
```
//Paso 2 //Grafo auxiliar
```
Grafo<entero> $A \leftarrow inicializarGrafo();$
para cada $w \in Vertices(G)$
  $agregarVertice(A, w)$
fin para
para cada $v' \in Adyacentes(G, v)$
  $agregarArista(A, v, v', Peso(G, v, v'))$
fin para
```
//Conjunto de Vértices pendientes de calculo
```
$Pendientes \leftarrow Vertices(G) \setminus Visitados$

Greedy Grafos – Dijsktra - Algoritmo

```
//Paso 3
mientras Pendientes ≠ ∅
  w ← n : PesoArista(A, v, n) = mín{PesoArista(A, v, p)}∀n, p ∈
  Pendientes
  Visitados ← Visitados ∪ {w}
  Pendientes ← Pendientes \ {w}
  auxPendientes ← Pendientes
  mientras auxPendientes ≠ ∅
    p ← elegir(auxPendientes)
    auxPendientes ← auxPendientes \ {p}
    si existeArista(A, v, w) Y existeArista(G, w, p)
      si existeArista(A, v, p)
        si pesoArista(A, v, w) + pesoArista(G, w, p) <
        pesoArista(A, v, p)
          agregarArista(A, v, p, pesoArista(A, v, w) +
          pesoArista(G, w, p)
        fin si
      sino
        agregarArista(A, v, p, pesoArista(A, v, w) + pesoArista(G, w, p)
      fin si
    fin si
  fin mientras
fin mientras
devolver A
```

Greedy Grafos – Dijsktra – Complejidad Temporal

ALGORITMO DIJKSTRA

Entrada: G: Grafo<entero>, v: entero

Salida: A Grafo<entero>

```
//Paso 1 //Conjunto de Vértices ya visitados
Conjunto<entero> Visitados ← {v}
//Paso 2 //Grafo auxiliar
Grafo<entero> A ← inicializarGrafo();
para cada w ∈ Vertices(G)
  agregarVertice(A, w)
fin para
para cada v' ∈ Adyacentes(G, v)
  agregarArista(A, v, v', Peso(G, v, v'))
fin para
//Conjunto de Vértices pendientes de calculo
Pendientes ← Vertices(G) \ Visitados
```

Cada ciclo depende de la cantidad de nodos o vértices, y los adyacentes, que en el peor de los casos itera en nodos – 1, por lo tanto la complejidad temporal es:

$$\mathcal{O}(n)$$

Nota: se consideran las complejidades de las operaciones sobre grafos constante, con una matriz de acceso directo.

Greedy Grafos – Dijsktra – Complejidad Temporal

```
//Paso 3
mientras Pendientes ≠ ∅
  w ← n : PesoArista(A,v,n) = mín{PesoArista(A,v,p)}∀n,p ∈
  Pendientes
  Visitados ← Visitados ∪ {w}
  Pendientes ← Pendientes \ {w}
  auxPendientes ← Pendientes
  mientras auxPendientes ≠ ∅
    p ← elegir(auxPendientes)
    auxPendientes ← auxPendientes \ {p}
    si existeArista(A,v,w) Y existeArista(G,w,p)
      si existeArista(A,v,p)
        si pesoArista(A,v,w) + pesoArista(G,w,p) <
        pesoArista(A,v,p)
          agregarArista(A,v,p,pesoArista(A,v,w) +
          pesoArista(G,w,p)
        fin si
      sino
        agregarArista(A,v,p,pesoArista(A,v,w) + pesoArista(G,w,p)
      fin si
    fin si
  fin mientras
fin mientras
devolver A
```

Dos ciclos anidados que dependen de la cantidad de nodos, por lo tanto iteran n veces, siendo n la cantidad de vértices. Dado lo anterior la complejidad temporal es:

$$\mathcal{O}(n^2)$$

Qué es
un árbol de recubrimiento

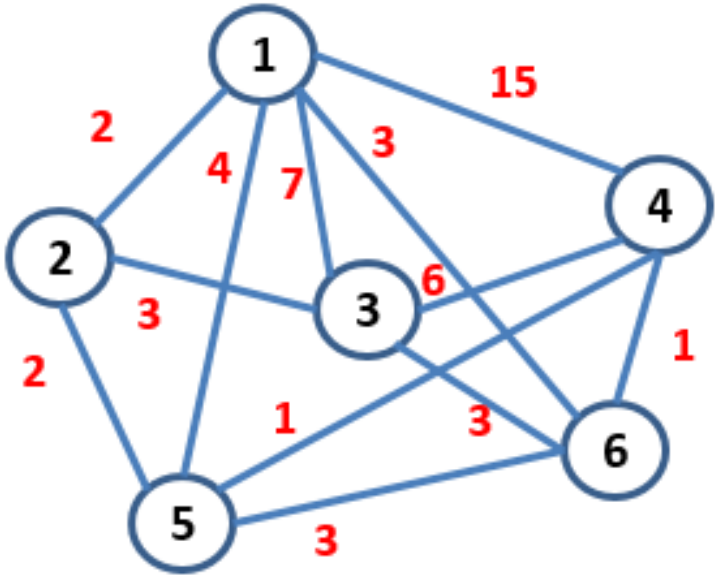
UADE

Árbol de recubrimiento de costo mínimo

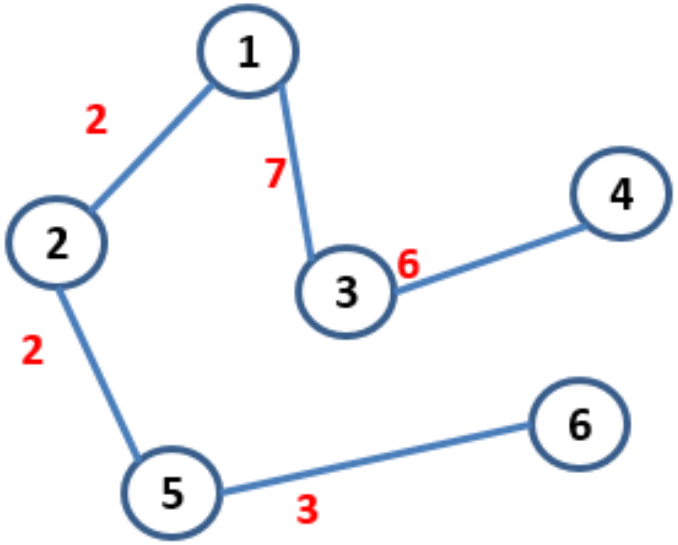
- Los árboles de recubrimiento se trabajan con grafos no dirigidos.
- Un árbol de recubrimiento de un grafo *G(V;A)* a un grafo *G'(V;A')* que contiene los mismos vértices que *G* y un subconjunto de las aristas, que forma un árbol, es decir, *G'* es un grafo conexo y acíclico.
- Árbol de recubrimiento mínimo es al árbol de recubrimiento que tenga el menor costo de todos, es decir que la suma de sus aristas sea la mínima.
- Para algunos grafos, puede existir más de un árbol de recubrimiento mínimo, todos con el mismo costo.

Árbol de recubrimiento Mínimo – Ejemplo 1

Grafo original



Ej. árbol recubrimiento



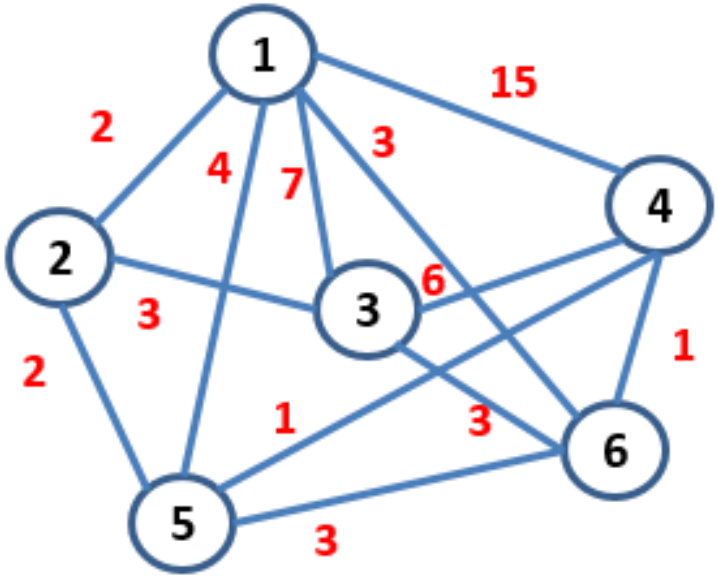
Costo: 20

No es el árbol de recubrimiento de costo mínimo

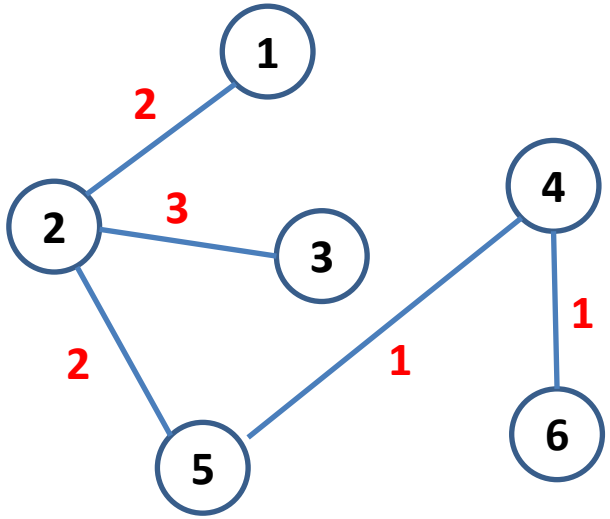
UADE

Árbol de Recubrimiento Mínimo – Ejemplo 2

Grafo original



Ej. árbol recubrimiento



Costo: 9

Es el árbol de recubrimiento de costo mínimo

UADE

Árbol de Recubrimiento Mínimo - Algoritmos

- Existen dos algoritmos con estrategia ***Greedy*** que permite obtener un árbol de cubrimiento de un grafo con costo mínimo.

- Los algoritmos son:
  - **PRIM** **$O(n^2)$, siendo n la cantidad de nodos**
  - **Kruskal** **O(a log a), siendo a la cantidad de aristas**

- En el caso de que se tenga un único árbol de recubrimiento de costo mínimo, ambos algoritmos arrojan el mismo resultado. Si no, podría variar por la estrategia que sigue cada uno.

Ejercicios algoritmos grafos

Ejercicios a Resolver

Lea detenidamente cada problema, por cada uno de ellos deberá responder los siguientes puntos:

d) Determinar **qué algoritmo sobre grafos aplicaría**.

e) Pseudocódigo del Algoritmo de la resolución del problema, el cual debe seguir la estrategia definida en el punto a)

f) Cálculo de la Complejidad Temporal, justificando la misma en forma detallada.

1. Dado un grafo G dirigido, cuyos nodos representan las escuelas y gimnasios de un barrio en forma numérica, y sus aristas la distancia en cuadras para llegar entre los diferentes lugares representados en el grafo. Determinar cuántas son las cuadras mínimas para llegar de la escuela **X** al gimnasio **Y** (ambos puntos están representados en el grafo).

¡Muchas gracias!

UADE